

本节介绍与本书配套的实验内核 ChCore 中的进程间通信机制的设计。在 ChCore 的通信机制中，消息的传递和通知基于 LRPC 中的迁移线程技术和 L4 系列微内核中的直接进程切换技术，而数据的传输则是基于灵活的用户态共享内存。

代码片段 8.16 给出的是 ChCore 进程间通信的用户态样例代码，主要包括客户端进程代码（第 21 ~ 32 行）和服务端进程代码（第 1 ~ 19 行）。可以看到服务端进程通过 `ipc_register_server` 向内核注册一个服务。这里的关键参数为服务的逻辑处理函数，即代码中的 `ipc_dispatcher`。当一个请求被发出后，服务端的逻辑处理函数被调用。其首先获取一个消息中数据的地址（第 6 行），并且在处理完逻辑后通过 `ipc_return` 将结果返回客户端进程。客户端进程通过 `ipc_register_client` 注册一个进行通信的结构体，这里其实是通过 ChCore 中的 Capability 子系统来完成客户端和服务端进程之间连接的建立。当一切准备工作就绪后，客户端进程通过 `ipc_call` 发起一次远程调用的通信，而参数会被放在 `ipc_msg` 中。消息结构体（`ipc_msg_t`）中既可以传递数据，也可以传递 Capability。因此，可以通过传递内存内核对象的 Capability 的方式，在两个进程间建立一个共享内存区域，用作大量数据的传输。

代码片段 8.16 ChCore 进程间通信的用户态样例代码

```

1 // 服务端：用户态 IPC 逻辑处理
2 void ipc_dispatcher(ipc_msg_t *ipc_msg)
3 {
4     u64 ret;
5     // 获取消息中的数据
6     char* data = ipc_get_msg_data(ipc_msg);
7     // 执行服务代码
8     ...
9     // 返回结果给客户端进程
10    ipc_return(ret);
11 }
12
13 // 服务端进程代码
14 void server()
15 {
16     ...
17    ipc_register_server(ipc_dispatcher);
18    ...
19 }
20
21 // 客户端进程代码
22 void client()
23 {
24     int server_process_cap;
25     ipc_struct_t client_ipc_struct;
26     ipc_msg_t *ipc_msg;
27     ...
28     // 获得服务端进程的权限

```

```

29  ipc_register_client(server_process_cap, &client_ipc_struct);
30  ...
31  // 发起 IPC
32  u64 ret = ipc_call(&client_ipc_struct, ipc_msg);
33  ...
34  }

```

在用户态的示例代码中，和进程间通信相关的关键路径上的两个系统调用接口是 `ipc_call` 和 `ipc_ret`，这对应着代码片段 8.17 中的两个 ChCore 系统调用 `sys_ipc_call` 和 `sys_ipc_return`。可以看到，在拿到内核中的通信对象（`conn`）之后，内核就通过 `thread_migrate_to_server` 接口将控制流直接转交给服务端进程（客户端进程是类似的）。这里的思路和 LRPC 以及 L4 中直接进程切换的思路类似，对于通信的情况绕开内核调度，直接完成上下文切换。

代码片段 8.17 ChCore 进程间通信的系统调用接口

```

1  u64 sys_ipc_call(u32 conn_cap, ipc_msg_t *ipc_msg)
2  {
3      // 获取 IPC 连接
4      struct ipc_connection *conn = get_connection(conn_cap);
5      // 处理 ipc_msg，如传递 Capability 等
6      ...
7      // 将控制流移交给服务进程，arg 为传递给服务端的参数
8      thread_migrate_to_server(conn, arg);
9
10     BUG("This function should never return!\n");
11     ...
12 }
13
14 void sys_ipc_return(u64 ret)
15 {
16     // 获取 IPC 连接
17     struct ipc_connection *conn = get_current_connection();
18     ...
19     // 控制流从服务进程转移回客户进程，ret 为返回给客户端的结果值
20     thread_migrate_to_client(conn, ret);
21
22     BUG("This function should never return!\n");
23 }

```

8.8 案例分析：Binder IPC

本节主要知识点

- ❑ Android 场景对于进程间通信有什么特殊的需求？
- ❑ Binder IPC 中的通信模型是如何设计的？

□ Binder IPC 中的命名服务是如何设计的？

□ Binder IPC 如何实现数据流的传输和控制流的转移？

8.8.1 总览

Android 是主要应用在手机等移动平台的操作系统，由 Linux 内核及一系列用户态服务组成。Android 用户态服务包括应用启动、屏幕绘制等重要功能。大部分 Android 应用进程需要依赖这些用户态服务的功能去实现应用逻辑。这就使得 Android 比一般宏内核系统对进程间通信有更高的要求。

Android 借鉴了微内核 IPC 的设计方法，针对移动端场景对 Linux 内核中通信的设计进行了优化，即 Binder IPC 机制。相比 Linux 的通信机制，Binder IPC 进程间通信机制在性能、接口易用性上都有提升。Binder IPC 涉及从底层的内核 Binder 驱动到用户态 Binder 接口，本节将围绕内核中的 Binder 设计进行介绍。

接口方面，Binder IPC 同样采用消息接口，并且同时支持同步和异步两类通信以支持不同的应用场景需求。本节将讨论 Binder IPC 的远程方法调用（RPC）设计，即服务端进程负责提供具体的服务，客户端进程则通过进程间通信发起服务请求并获得服务端进程处理后的结果。本节将首先介绍 Binder IPC 的通信连接和命名服务的设计，然后介绍基于线程池的服务端响应模型、基于该模型的通信过程以及 Binder IPC 对数据传递的优化设计。

8.8.2 Binder IPC 内核设计

通信连接与命名服务

除了通信双方的进程外，Binder IPC 中引入了一个 Context Manager^①进程。Context Manager 提供命名服务，它的任务是建立通信连接。Binder IPC 的内核设计中提供了句柄（Handle）的抽象，作为 IPC 对象（即一个通信连接）的表示。句柄和文件描述符很相似，用户通过对句柄的操作来发起对特定进程的通信。客户端进程和服务端进程启动后，会分别先和 Context Manager 进程建立连接，如图 8.12 中的通信连接①和②。

Context Manager 进程在初始化时，会通过内核提供的一个特殊的命令接

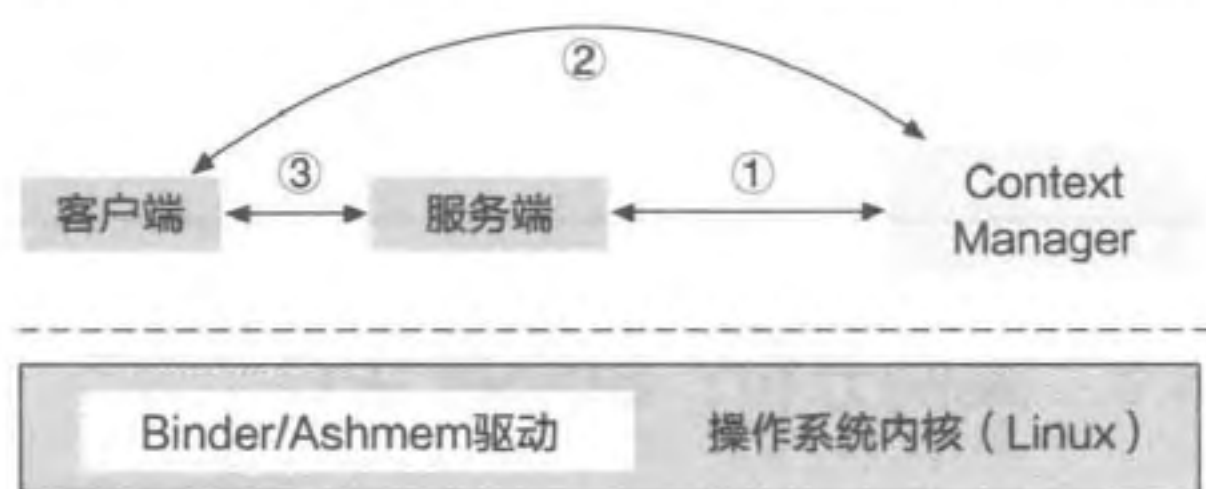


图 8.12 Binder 通信连接

① Context Manager 在更高层的 Android 抽象中也叫作 Service Manager，这里为了统一，只使用 Context Manager 的称呼。

□ `BINDER_SET_CONTEXT_MGR`，声明自己是 Context Manager (全局唯一的)。内核令 Context Manager 对应的句柄为常量 0，并且允许任何进程和 0 号句柄代表的 Context Manager 通信。这样一来，只要一个进程完成基本的初始化，就可以和 Context Manager 通信并进一步注册服务或是和某个服务建立连接。

Context Manager 的两个基本功能是：让服务端注册服务 (`SVC_MGR_ADD_SERVICE`)，以及让客户端连接特定的服务端服务 (`SVC_MGR_GET_SERVICE`)。这两个操作中，除了基本的元数据的注册和查询 (如服务名等) 之外，还包含十分重要的句柄转发 (和微内核系统中的 Capability 转发类似)。Binder IPC 的消息中，除了数据，还允许传递其他类型的信息，如文件描述符、IPC 句柄等。内核负责将文件描述符、IPC 句柄等特殊的信息传递给目标进程。此处的目标进程在通信发起时是服务端进程，在返回结果时是客户端进程。在服务端调用注册服务接口 `SVC_MGR_ADD_SERVICE` 时，它会将服务的基本信息和对应的句柄通过 Binder 通信发送给 Context Manager。此处的句柄是服务端事先向内核注册该服务时创建的。而当客户端进程通过 `SVC_MGR_GET_SERVICE` 建立连接时，Context Manager 会将对应的服务句柄返回客户端，如图 8.12 中的通信连接③。这样，内核不需要提供额外的建立连接的机制，只需要完成 IPC 消息的传递就可以让 Context Manager 来管理通信连接。

相比宏内核中的 IPC 方案，Binder 命名服务的设计和微内核 IPC 更像。通过用户态的 Context Manager 动态建立两个进程之间的连接，相比由内核来建立连接更加灵活，并且系统设计者能够在 Context Manager 中实现更加复杂的连接管理功能。此外，该设计还能够减少对内核的修改，内核中只需要提供必要的支持，如记录全局唯一的 Context Manager 等。

关键设计：线程池模型

和之前的进程间通信设计不同的一点是，Binder IPC 采用“线程池”的服务端模型。即在服务端，Binder 的用户态和内核有一个响应线程池的概念。当某个客户端进程发起通信时，内核会从 (服务端的) 线程池中选择一个可用的线程来响应。这种设计能够在同步进程间通信的情况下比较好地处理并发的通信请求。

在 Android 场景下，一个服务端进程可能会面对大量客户端进程，且多个进程可以同时调用该服务。为了更高效地处理客户端请求，服务端进程在向内核以及 Context Manager 注册信息时，内核 Binder 驱动及服务端进程会维护一个线程池。线程池中的每个线程都可以处理客户端的请求并返回结果。这些线程需要自己处理并发情况下的同步和正确性问题。内核将具体的执行线程抽象为 `Looper`，服务端进程通过 `BC_REGISTER_LOOP` 接口向内核注册一个 `Looper`。当客户端进程通过句柄发起通信时，内核在句柄对应的服务端进程中找到空闲的 `Looper` 来响应请求。

简单的线程池模型存在权衡问题，即如果事先分配过少的处理线程，那么很难支持高并发场景，而如果事先分配过多的线程，又可能对系统资源造成浪费。Binder IPC 支

持在请求到来时动态地创建新的处理线程，从而解决这个问题。Binder 驱动允许用户配置一个最大的线程数，当客户端进程请求通信时，若内核发现当前没有可用的 Looper，则内核会检查此时是否能够创建新的线程。如果当前内核请求创建的线程数没有超过配置的最大值，那么内核通知服务端进程，让其创建新的线程来处理请求。动态分配结合可定义的最大线程数，可以有效防止资源浪费以及潜在的攻击。

通信过程

下面介绍 Binder IPC 围绕线程池模型的通信过程，包括以下问题：首先，服务端进程的处理线程是如何“等待”在内核中的；然后，客户端进程如何通过 Binder 事务发起一次远程通信，以及服务端进程如何响应并且回复；最后，在通信过程中，客户端和服务端能够交换的消息类型包含哪些。

服务端处理线程

在 Binder 通信中，第一步需要服务端准备好相关的处理线程来响应客户端的请求。如图 8.13 所示，Binder 协议提供了两个命令来注册一个处理线程：BC_REGISTER_LOOPER 和 BC_ENTER_LOOPER。这两个命令之间有什么区别，分别在什么场景下使用呢？首先，BC_ENTER_LOOPER 是服务端进程主动发起的向线程池中注册处理线程的命令，这不会受到前文介绍的最大线程数的限制，服务端进程可以注册任意多个。与之对应，当有请求到达服务时，内核驱动如果发现当前没有可以使用的处理线程，内核会通知服务端进程注册额外的处理线程，而由内核通知进行注册的处理线程是用 BC_REGISTER_LOOPER 来实现的。这种方式注册的处理线程会受到配置的最大线程数的约束。使用两个接口的一个原因是，内核可以维护当前由内核要求注册的线程数，检查其是否超过最大线程数的限制，从而避免因恶意的客户端进程大量发起通信而导致内核申请创建过多的处理线程。

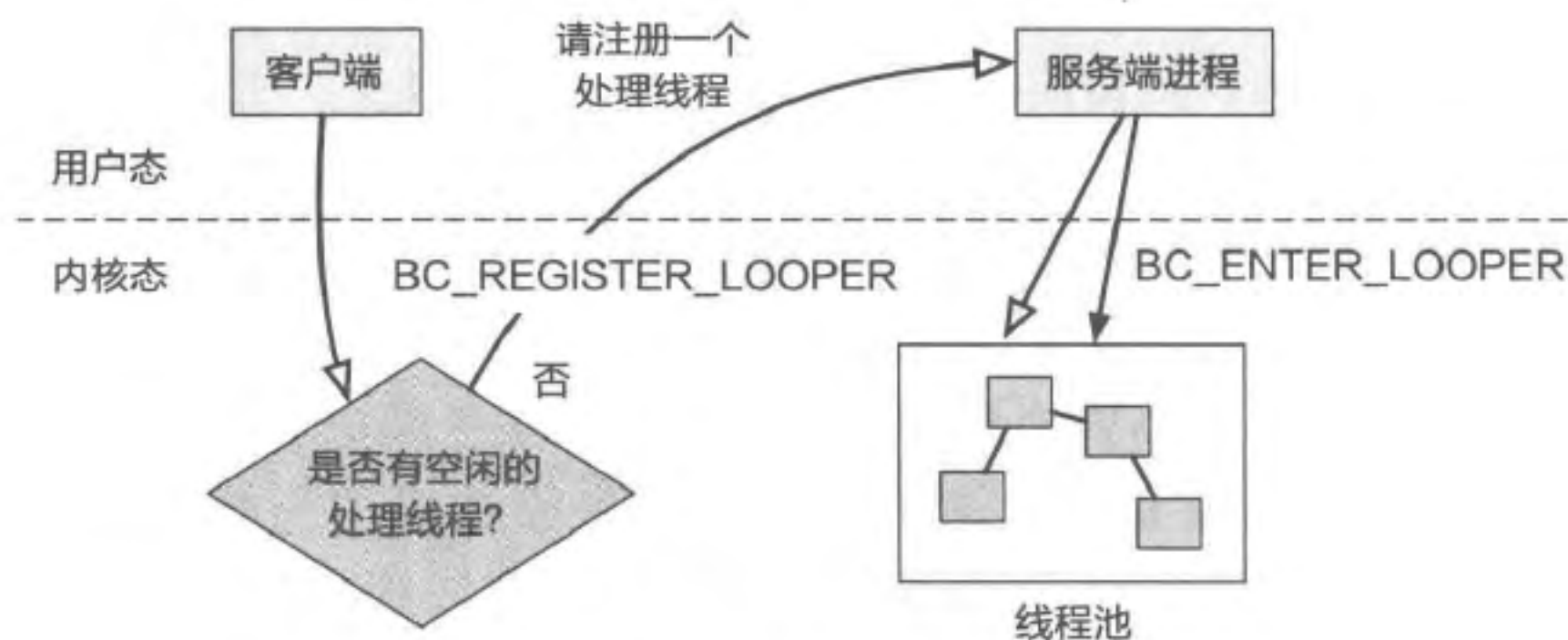


图 8.13 注册服务端处理线程

Binder 事务

Binder IPC 将通信的发起和返回也叫作 Binder 事务（Transaction）。在一次正常的同步通信中，Binder 涉及两个命令 BC_TRANSACTION 和 BC_REPLY，以及对应的两

个返回值。当客户端进程请求服务端进程执行服务时，客户端进程会向内核驱动发起一次 `BC_TRANSACTION` 命令^①，这个命令即代表通信的发起，内核 Binder 驱动会将对应的数据和请求发送到目标服务进程来处理。服务端进程处理完请求后，会使用命令 `BC_REPLY` 来请求内核驱动将处理的数据结果返回客户端进程。内核将 `BC_TRANSACTION` 的通信请求发送到服务端进程时，需要将一个阻塞在内核中的处理线程唤醒（如果此时没有可用的，会先让服务端进程创建处理线程）。该处理线程在被唤醒后得到一个返回值 `BR_TRANSACTION`，表示有新的请求需要处理。当结果返回给客户端时，内核会创建返回值 `BR_REPLY`。从客户端程序的视角来看，其发出的请求已经处理完毕，并且获得了处理结果。当然，上述过程假设通信过程一切顺利。如果遇到异常情况，如服务端进程已经退出等，内核会通过 `BR_ERROR` 等表示错误的返回值来通知通信的进程。

消息类型

在通信的发起和回复过程中，通信的消息是怎样设计的呢？

Binder 消息除了可以传输普通的数据外，还可以传输 Binder 句柄，甚至是通用的文件描述符。这使得两个进程间可以通过通信动态建立新的连接（即前文 Context Manager 工作的基础）。图 8.14 展示了 Binder 消息的内部组成（简化版本）。其中，以 `buffer` 开始，长度为 `data_size` 的区间为通信的数据。而在 `data_size` 之后，长度为 `offset_size` 的区间为偏移数据。为什么需要偏移数据呢？这是因为在 Binder 传输的消息中，普通数据和特殊数据（Binder 引用、文件描述符等）是混在一起的，都在 `buffer` 指向的区间内。

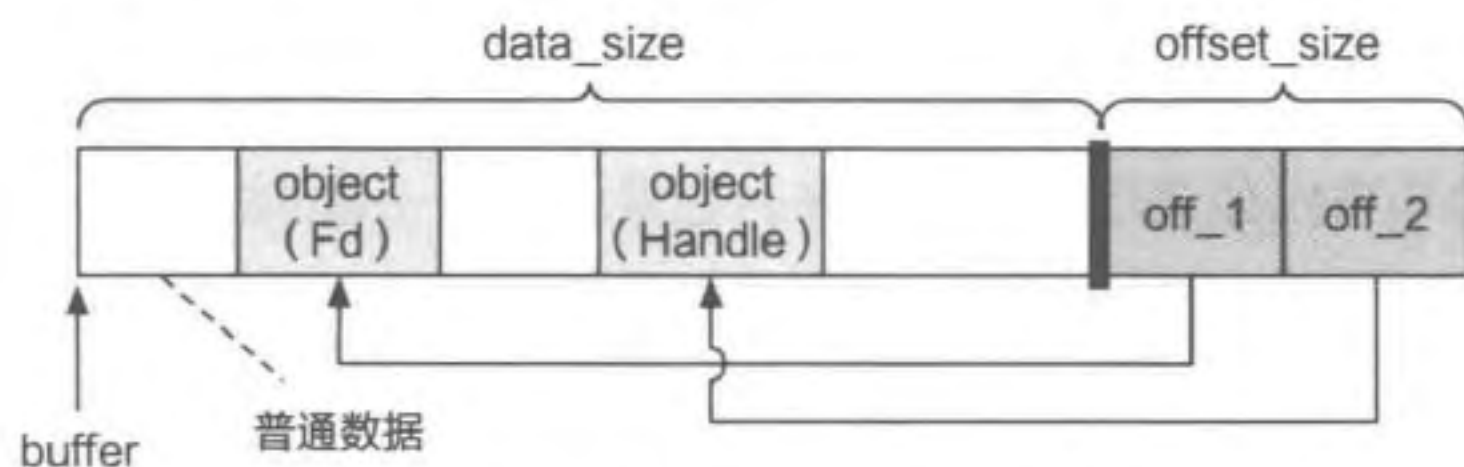


图 8.14 Binder 的通信数据布局

特殊数据是需要内核特殊处理的。比如，对于一个文件描述符类型的数据，内核会检查该文件描述符是否存在，然后将其对应的文件打开在服务端进程中（此时得到在服务端进程中的文件描述符），最终将服务端的文件描述符返回服务端进程。因此，服务端进程可以直接使用该文件描述符。在引入偏移数据前，内核可能需要扫描整个数据段来识别特殊数据并进行处理，这显然是很耗时的。

偏移数据能够有效解决这个问题。如图 8.14 所示，偏移数据中的每一项对应一个 `buffer` 中的偏移位置，表示这个位置的数据是特殊的类型，需要 Binder 驱动专门处

① 本节介绍的大部分用户态向内核发起的命令是通过 `ioctl` 接口完成的。

理。其中，`offset_size` 表示有多少项。通过这种方式，内核驱动可以高效地定位到每一个特殊数据，从而根据它们的类型进行处理。

8.8.3 匿名共享内存

Android 使用匿名共享内存（Ashmem）的共享内存机制来传递大量的数据，并将其结合在 Binder IPC 的使用当中。

为什么需要匿名共享内存

在 Linux 系统中，有两种现成的共享内存机制：一种是 `mmap` 系统调用映射的共享内存区域，它允许父子进程之间共享这段内存；另一种是我们前面介绍的 System V 的共享内存。第一种方式受限于共享内存的进程间必须存在相应的父子等关系，在实际系统中很难（动态）建立映射。而 System V 的共享内存机制通过 IPC key 来共享一段内存。但是这里的共享粒度是根据进程是否有对应的用户/组权限，以及是否知道对应的 key 来确定的。这就导致很多时候权限过宽，不应该访问这段共享内存的进程也可能通过尝试不同的 key 来映射共享内存。匿名共享内存的提出正是为了解决上面的两个问题。Android 希望利用匿名共享内存存在 Binder 通信的双方向非常灵活地建立共享内存，而不需要父子关系以及事先知道某个 key。

基于文件描述符的共享内存设计

与 System V IPC 共享内存相比，匿名共享内存改变了使用和共享的方式，如图 8.15 所示。匿名共享内存使用文件描述符而非全局的 key 来定位具体的共享内存，并且在通常情况下也没有其他全局的名字。从图中可以看到，两个进程为了共享一个匿名共享内存区域，需要各自有一个文件描述符，指向同一个文件结构（file）。和共享内存中相同的，这个文件是一个特殊的共享内存文件系统的文件，它最终指向一段页缓存区域中的物理内存页。

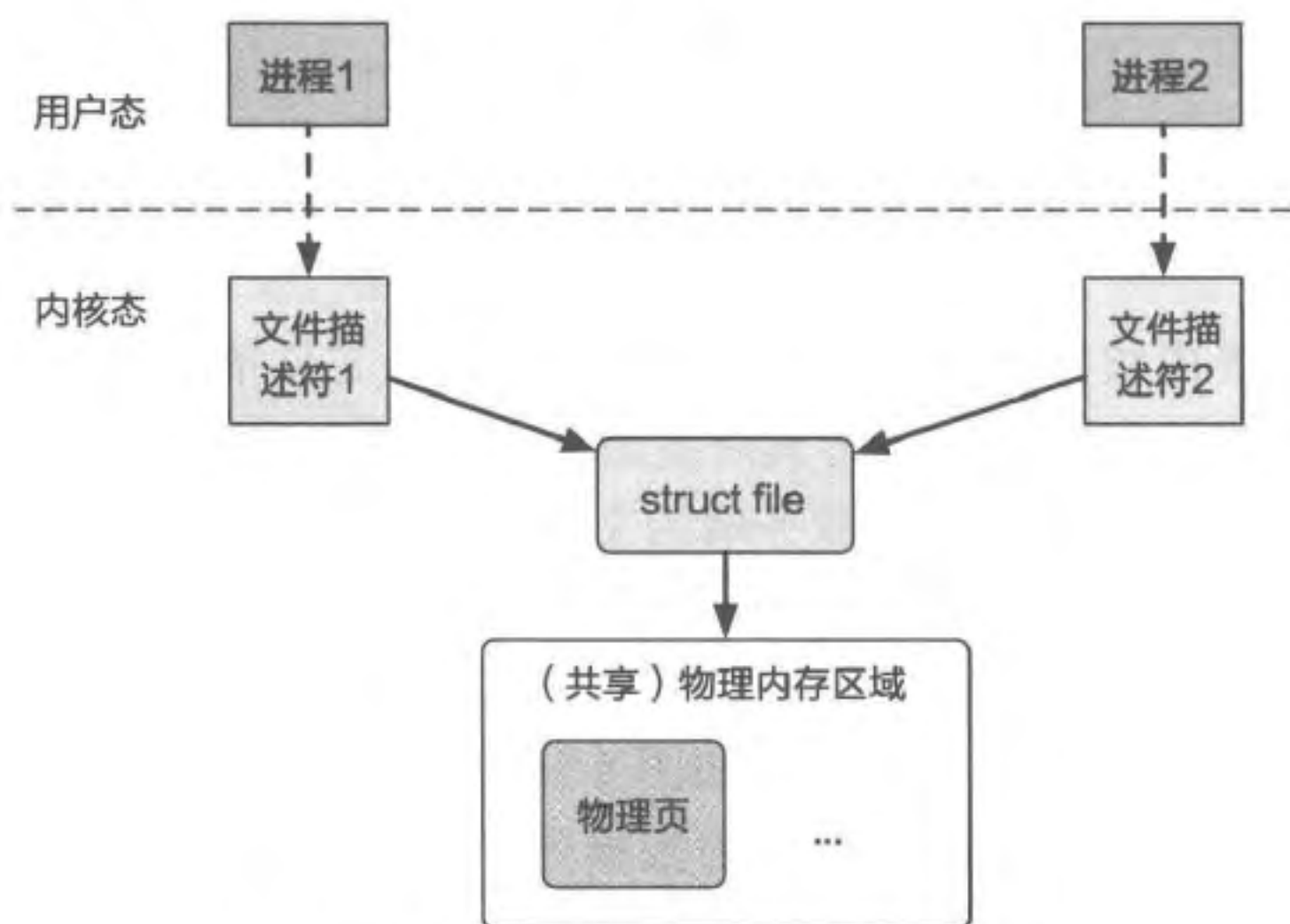


图 8.15 基于文件描述符的匿名共享内存

使用匿名共享内存时，首先，进程 1 打开匿名共享内存对应的字符设备，设置内存段的大小，然后通过 `mmap` 映射文件的方式，将创建的匿名共享内存映射到用户态空间。完成这几步操作，进程 1 就可以像使用普通内存一样，在匿名共享内存上读写数据了。进程 2 如果希望使用该匿名共享内存，需要进程 1 通过传递文件描述符的方式，将描述符传递给进程 2，使其拥有对该内存的访问权限。

结合 Binder 的共享内存传输

前文提到 Binder 的消息是可以传递文件描述符的，这就使得进程间可以通过 Binder IPC 机制灵活地传递匿名共享内存。通信的两个进程中，其中一个进程可以在消息中放置匿名共享内存对应的文件描述符。Binder 内核驱动在处理消息数据时，会自动在另一个进程的文件描述符列表中打开一个新的描述符，让它指向这个匿名共享内存对应的文件，即建立图 8.15 中的关系。相当于进程可以在发起通信的同时，通过匿名共享内存快速传递大量数据，这可以极大地优化使用 Binder 来传递数据的性能。

8.9 思考题

1. 进程间通信的性能和硬件体系结构设计息息相关。一个例子是权限级的切换，大部分进程间通信的实现需要进入内核操作，即会发生用户态和内核态之间的权限级切换。这会带来不小的开销。请列出你觉得会对进程间通信性能造成影响的其他硬件设计，并解释造成开销的原因和可能的优化方法。
2. 基于共享内存的进程间通信有较好的数据传输能力。然而在不拷贝数据的情况下，共享内存的一个问题是“检查到使用”（Time-of-Check to Time-of-Use）窗口攻击。简单来说，服务端通常会先检查共享内存上的数据（Time-of-Check），检查通过后再使用数据（Time-of-Use）。然而，由于客户端仍然可以访问共享内存，因此可能在服务端检查之后、使用之前破坏或构造特定的数据来绕过检查，从而攻击服务端。在现有的硬件架构（x86 或者 AArch64）下，有什么办法可以在保留共享内存零拷贝的情况下避免这样的攻击？
3. 请给出一种管道设计，使其能够支持双向通信。
4. 我们介绍了 L4 的短消息和长消息的数据传输机制，在这两类消息之间的消息称为中等消息。由于长消息涉及内存重映射，通常需要以页（如 4KB）为单位。中等消息（比如 512B）无法放在短消息的寄存器中，采用长消息的重映射又会浪费内存。请设计一种中等消息传输机制，并解释你的设计。
5. L4 提出的惰性调度机制是为了避免在通信过程中访问调度队列。惰性调度对于调度的性能有怎样的影响？请给出你的分析。
6. 迁移线程的方案在多核场景下有什么优势，又有什么劣势？
7. 基于共享内存的数据传输方式往往需要对数据进行序列化，请分析原因。